

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра «ЭВМиС»

Калькулятор для работы с матрицами
ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе по дисциплине:
«Основы алгоритмизации и программирования»
КР.Э-64/25.250781 – 01 81 00

Листов: 29

Разработал:

Шрейдер А.А.

Проверил:

Мешечек Н.Н.

Содержание

ВВЕДЕНИЕ.....	4
1 ВЫБОР МЕТОДА РЕАЛИЗАЦИИ ЗАДАЧИ.....	5
1.1 Оценка предметной области и выбор метода решения задачи.....	5
1.2 Выделение и обоснование функциональных частей.....	7
1.3 Составление общего алгоритма.....	9
2 РАЗРАБОТКА ТЕСТОВЫХ ПРИМЕРОВ.....	11
3 РАЗРАБОТКА ПРОГРАММЫ.....	14
4 ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ.....	19
ЗАКЛЮЧЕНИЕ.....	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	29

ВВЕДЕНИЕ

Целью курсовой работы является разработка консольной программы для выполнения математических операций над матрицами (сложение, вычитание, умножение, умножение на скаляр, транспонирование).

Была разработана консольная программа «Калькулятор матриц» на языке C. Для удобства в программе реализовано меню с выбором пунктов.

Проект построен на структурном программировании: каждая математическая операция (сложение, вычитание, умножение матриц, транспонирование и умножение на скаляр) вынесена в отдельную изолированную функцию. При разработке были использованы все ключевые возможности стандартного C: условные операторы, циклы, массивы и библиотеки консольного ввода-вывода [1].

В калькуляторе предусмотрены разные возможности ввода матрицы: пользователь может вводить элементы матрицы вручную, создавать единичные или нулевые матрицы, а также использовать автоматическое заполнение случайными числами. Перед выполнением любых математических действий программа проводит строгую проверку данных. Например, алгоритм проверяет равенство размеров матриц при сложении и вычитании, а также согласованность размерностей (число столбцов первой матрицы должно быть равно числу строк второй).

Актуальность данной курсовой работы обусловлена тем, что матричные вычисления лежат в основе большинства задач. Инженерные расчёты, обработка цифровых изображений, трехмерная компьютерная графика базируются на линейной алгебре.

Практическая значимость работы заключается в приобретении реального опыта разработки законченного программного продукта в консольном режиме.

1 ВЫБОР МЕТОДА РЕАЛИЗАЦИИ ЗАДАЧИ

1.1 Оценка предметной области и выбор метода решения задачи

Работа с матрицами является основой вычислительной математики и современного программирования. Матрица представляет собой прямоугольную таблицу чисел, структурированную по строкам и столбцам [2,3]. В программировании матрицы выступают в качестве структуры данных, предназначенной для хранения, упорядочивания и высокоскоростной обработки числовой информации.

Область применения матриц чрезвычайно широка [3]:

- При проектировании сложных технических систем, расчете прочности конструкций и анализе электрических цепей возникают системы линейных алгебраических уравнений (СЛАУ), содержащие тысячи переменных. Матричный аппарат позволяет эффективно решать эти системы [2].

- В двумерной и трехмерной графике любые преобразования объектов (поворот, масштабирование, сдвиг, проецирование 3D-моделей на плоский экран) реализуются посредством перемножения матриц координат на матрицы трансформации.

Применение матриц в этих сферах обусловлено тем, что они позволяют упаковать огромные массивы в единый объект и применять к нему универсальные математические законы. Вместе с тем, использование матричного подхода имеет свои достоинства и ограничения.

К основным преимуществам применения матриц относятся:

- *Высокая степень структурированности данных*: информация хранится в интуитивно понятной табличной форме с мгновенным доступом к любому элементу по его индексам.

- *Возможность векторизации и распараллеливания*: матричные операции идеально подходят для ускорения вычислений на современных многоядерных процессорах и видеокартах.

К недостаткам и техническим сложностям можно отнести:

- *Нелинейный рост вычислительной сложности*: при увеличении размерности матрицы количество необходимых операций возрастает экспоненциально.

- *Необходимость строгой проверки*: большинство матричных операций требуют точного совпадения размерностей операндов, что вынуждает программиста вводить дополнительные проверки.

Разработанная программа «Калькулятор матриц» предназначена для автоматизации базовых матричных вычислений в консольном режиме. Выбор языка программирования обусловлен тем, что С является стандартом для изучения структур данных [1]. Он обеспечивает прямой доступ к памяти компьютера, позволяет управлять индексацией массивов и не перегружен скрытыми механизмами.

Преимущества языка С в рамках данной задачи: максимальная скорость выполнения скомпилированного кода, простота синтаксиса, эффективная работа с двумерными массивами, расположенными в памяти непрерывным блоком [1].
Недостатки языка С: отсутствие автоматической сборки мусора и встроенного контроля выхода за границы массива, что требует от разработчика повышенной аккуратности при проверке индексов.

Для решения поставленной задачи выбран метод обработки двумерных массивов с использованием вложенных циклов. В основе программы лежат следующие базовые алгоритмы:

- *Сложение и вычитание:* реализуются через двойной вложенный цикл (по строкам и столбцам)[2]. Выполняется поэлементная обработка $C_{ij} = A_{ij} \pm B_{ij}$ $C_{ij} = A_{ij} \pm B_{ij}$.

- *Умножение матриц:* реализуется с помощью трех вложенных циклов. Каждый элемент результирующей матрицы рассчитывается как скалярное произведение строки первой матрицы на столбец второй: $C_{ij} = \sum (A_{ik} \cdot B_{kj})$ $C_{ij} = \sum (A_{ik} \cdot B_{kj})$ [2,5].

- *Транспонирование:* зеркальное отражение элементов относительно главной диагонали, где $A_{ij}^T = A_{ji}$ $A_{ij}^T = A_{ji}$.

- *Умножение на скаляр:* линейный проход по всем элементам с умножением каждого на заданное число.

1.2 Выделение и обоснование функциональных частей

Для высокой читаемости кода, простоты отладки разрабатываемая программа построена на принципах структурного программирования [1]. Код программы разделен на изолированные функции, каждая из которых решает одну конкретную подзадачу.

Все функции программы можно разделить на четыре группы: интерфейсные функции, функции ввода и инициализации данных, функции математической обработки и функции вывода.

а) Интерфейсные функции управлением навигации

1) Функция `info()`. Отвечает за приветствие пользователя. При запуске программы она выводит на экран информацию о названии курсовой работы, а также краткую справку о математических операциях и правилах навигации.

2) Функция `menu()`. Функция выводит интерактивное меню в консоли. Вместо ручного ввода цифр пунктов меню, здесь реализовано управление через клавиши клавиатуры (стрелки «Вверх» и «Вниз»). Подтверждение выбора осуществляется клавишей `Enter`.

3) Функция `pause_screen()`. Выполняет роль синхронизатора. Поскольку консольные программы склонны мгновенно очищать экран перед следующим шагом цикла, эта функция останавливает выполнение программы сообщением «Нажмите любую клавишу для продолжения...».

б) Функции ввода данных и инициализации

1) Функция `read_size()`. Модуль строгого контроля ввода. Запрашивает у пользователя количество строк и столбцов будущей матрицы. Введенные значения должны быть строго целыми положительными числами ($N > 0$) и не превышать константу `MAX`, чтобы избежать переполнения стека памяти. При вводе некорректных данных функция блокирует дальнейшее выполнение и требует повторного ввода.

2) Функция `create_matrix()`. Генератор матриц. В зависимости от выбора пользователя, модуль перенаправляет поток выполнения на один из четырех сценариев:

в) Функции математической обработки

1) Функция `slozhenie()`. Принимает на вход две исходные матрицы и матрицу-результат. Производит поэлементное сложение. Предварительно убеждается, что габариты матриц идентичны.

2) Функция `vichitanie()`. Архитектурно идентична функции сложения, но реализует операцию разности элементов.

3) Функция `umnozhenie()`. Наиболее сложный вычислительный модуль. Реализует классический алгоритм перемножения матриц $A_{m \times n}$ и $B_{n \times k}$ для получения матрицы $C_{m \times k}$. Функция содержит встроенный предохранитель: проверку равенства числа столбцов первой матрицы числу строк второй.

4) Функция `umnozhenie_skalar()`. Осуществляет проход по матрице с умножением каждого элемента на введенный пользователем скаляр (множитель).

5) Функция `transponirovanie()`. Выполняет замену индексов `[i][j][i]` `[j]` на `[j][i][j][i]`, перезаписывая элементы в новую матрицу.

6) 4. Функции вывода

7) Функция `print()`. Функция форматирует вывод числовых значений, преобразуя двумерный массив в ровную, легко читаемую таблицу в консоли.

8) Функция `main()`. Точка входа в программу. Выступает в роли главного диспетчера. Внутри `main()` объявлены исходные массивы данных и организован бесконечный цикл `while`, который вызывает меню, считывает код выбранной операции, запускает соответствующие функции инициализации, математики и вывода, а также обрабатывает команду завершения программы.

1.3 Составление общего алгоритма

Общий алгоритм работы программы «Калькулятор матриц» представляет собой циклическую структуру с ветвлением. Весь цикл программы от запуска до завершения можно разделить на несколько последовательных этапов.

Этап 1. Инициализация и старт

При запуске исполняемого файла операционная система передает управление функции `main()`. На этом этапе выделяется память под рабочие двумерные массивы. Программа вызывает функцию `info()` для вывода стартовой заставки. Затем запускается главный рабочий цикл приложения, и на экране появляется главное интерактивное меню (`menu()`).

Этап 2. Взаимодействие с пользователем и навигация

Пользователь видит список доступных операций: сложение, вычитание, умножение матриц, умножение на скаляр, транспонирование и выход. Программа переходит в режим ожидания прерывания от клавиатуры. При нажатии стрелок навигации курсор в консоли визуально смещается к выбранному пункту. Нажатие `Enter` фиксирует выбор и передает код операции в структуру `switch-case` главной функции. Если выбран пункт «Выход», цикл прерывается, память освобождается, и программа завершает работу с кодом 0.

Этап 3. Ввод

Если выбрана математическая операция, программа переходит к сбору исходных данных. Сначала вызывается модуль `read_size()`. Пользователю предлагается ввести количество строк и столбцов. Программа проверяет выполнение двух условий:

- 1) $M > 0$ и $N > 0$ (матрица не может быть нулевого или отрицательного размера).
- 2) $M \leq MAX$ и $N \leq MAX$ (защита от переполнения буфера).

Если проверка провалена, срабатывает обработчик исключений: выводится предупреждение, и программа возвращает пользователя к шагу ввода чисел.

Этап 4. Формирование матриц

После успешного подтверждения размеров вызывается `create_matrix()`. Пользователь выбирает метод заполнения:

- Если выбран ручной ввод, запускается двойной цикл, запрашивающий число для каждой координаты `[i][j]`.
- Если выбрано случайное заполнение, система инициализирует генератор псевдослучайных чисел и заполняет массив числами в заданном диапазоне.
- Если выбрана единичная матрица, алгоритм проверяет равенство $M == N$. Если пользователь запросил прямоугольную единичную матрицу, выводится сообщение об ошибке генерации, так как единичная матрица по законам линейной алгебры может быть только квадратной.

Этап 5. Проверка математической совместимости и вычисление

Перед тем как начать подсчет матриц, диспетчер `main()` производит вторичную проверку:

- При операциях сложения и вычитания проверяется равенство размерностей первой и второй матриц. Если матрицы разные, расчет не производится.

- При операции умножения двух матриц проверяется количество столбцов первой матрицы должно быть строго равно количеству строк второй. В противном случае вычислять произведение математически невозможно.

- Операции транспонирования и умножения на скаляр являются унарными и выполняются для матриц любых допустимых размеров без дополнительных проверок.

Если все проверки пройдены успешно, программа вызывает соответствующую вычислительную функцию (`slozhenie()`, `umnozhenie()` и т.д.), которая производит расчет и записывает итоговые числа в конечную матрицу `C`.

Этап 6. Финализация шага

После завершения расчетов вызывается функция `print()`, которая выводит на экран итоговую матрицу в виде отформатированной таблицы. Затем управление передается функции `pause_screen()`. Программа замирает, ожидая реакции пользователя. После нажатия любой клавиши консоль очищается, и программа возвращается на Этап 1, снова отображая главное меню и ожидая новых команд.

Подобный алгоритм гарантирует стабильность приложения: программа не «вылетает» при вводе некорректных символов, не позволяет выполнить математически запрещенные действия и обеспечивает комфортный циклический процесс работы.

2 РАЗРАБОТКА ТЕСТОВЫХ ПРИМЕРОВ

Разрабатываемая программа «Калькулятор матриц» будет предназначена для выполнения основных операций над матрицами в консольном режиме. Работа программы будет строиться на взаимодействии пользователя с интерфейсом через меню. После запуска программы пользователю будет выводиться информационное окно, содержащее название проекта, список поддерживаемых операций и описание элементов управления. После ознакомления с информацией пользователь может перейти к дальнейшей работе с программой.

Главное меню программы будет содержать список доступных операций над матрицами. Пользователь сможет выбрать сложение матриц, вычитание, умножение матриц, транспонирование, умножение матрицы на скаляр или завершение работы программы. Перемещение по пунктам меню будет выполняться при помощи клавиш вверх и вниз на клавиатуре, а выбор необходимого действия клавишей Enter. Такой способ управления позволит сделать работу программы более удобной и упростит навигацию между функциями.

После выбора операции программа предложит пользователю ввести размеры матриц. На данном этапе выполнится проверка корректности введенных значений. Размеры матриц должны быть положительными и не превышать установленного ограничения MAX, равного 50. Если пользователь введёт неверные данные, программа выведет сообщение об ошибке и предложит повторить ввод.

После ввода размеров пользователь выберет способ создания матрицы. В программе будет предусмотрено несколько вариантов заполнения матриц. Пользователь сможет вводить элементы вручную, создавать нулевые матрицы, единичные матрицы или заполнять матрицы случайными числами.

При ручном вводе пользователь поочередно будет вводить значения элементов матрицы. В случае выбора нулевой матрицы все элементы автоматически получат значение ноль. При создании единичной матрицы программа дополнительно проверит, является ли матрица квадратной [3]. Если количество строк и столбцов не совпадет, появится сообщение о невозможности выполнения данной операции. При случайном заполнении элементы матрицы будут генерироваться автоматически при помощи функции случайных чисел.

После создания матриц программа выполнит выбранную пользователем операцию. При сложении матриц каждый элемент первой матрицы будет складываться с соответствующим элементом второй матрицы. Для выполнения данной операции размеры матриц должны совпадать. Если размеры отличаются, программа выдаст сообщение об ошибке и возвратит пользователя к повторному вводу данных.

Во время тестирования операции сложения будут проверены полностью корректные, частично корректные и полностью неверные примеры. В корректном примере будут введены две матрицы одинакового размера, после чего программа

должна будет вывести успешный результат сложения. При частично корректном примере размеры матриц не будут совпадать, поэтому программа выведет сообщение об ошибке. В полностью неверном примере введем отрицательные размеры матрицы, в результате чего программа также должна будет сообщить о неверном вводе данных.

Операция вычитания матриц будет работать аналогичным образом. Каждый элемент второй матрицы будет вычитаться из соответствующего элемента первой матрицы. Для проверки данной функции также будем использоваться различные тестовые примеры. При вводе корректных данных программа должна правильно выполнить вычисления и вывести результат на экран. Если размеры матриц не совпадут, выполнение операции блокировать.

При тестировании умножения матриц особое внимание будет уделяться проверке совместимости размеров. Для выполнения умножения количество столбцов первой матрицы должно совпадать с количеством строк второй матрицы [5]. В корректном примере программа должна успешно выполнить вычисления и вывести результирующую матрицу. В случае несовместимых размеров программа должна вывести сообщение об ошибке и предотвратить выполнение операции. Также проведем тестирование с вводом размеров, превышающих допустимое ограничение. В такой ситуации программа должна вывести сообщение о неверном размере матрицы.

При проверке функции транспонирования будем использовать матрицы различных размеров. После выполнения операции строки матрицы будут становиться столбцами, а столбцы — строками [2]. Результаты тестирования должны показать корректную работу алгоритма транспонирования. Дополнительно проведем проверку создания единичной матрицы. Если пользователь попытается создать единичную матрицу для прямоугольной таблицы, программа выведет сообщение об ошибке.

Во время тестирования операции умножения на скаляр программа будет выполнять умножение каждого элемента матрицы на введенное пользователем число. Проверка должна показать, что операция выполняется корректно для различных значений скаляра и размеров матриц. Также проверим случаи неверного ввода размеров матрицы.

В программе будет предусмотрена обработка различных ошибок пользователя. Если пользователь введет неверные размеры матрицы, программа не позволит продолжить выполнение операции. При попытке выполнить невозможное действие, например сложение матриц разных размеров или умножение несовместимых матриц, программа выведет соответствующее сообщение об ошибке. После отображения сообщения программа будет ожидать нажатия клавиши и возвратит пользователя в меню или к повторному вводу данных.

Во время тестирования также проверим работу интерфейса программы. Будут протестированы навигация по меню, корректность переходов между экранами и работа управления с клавиатуры. Проверка должна показать, что программа корректно реагирует на действия пользователя и выполнит переходы

между функциями без ошибок.

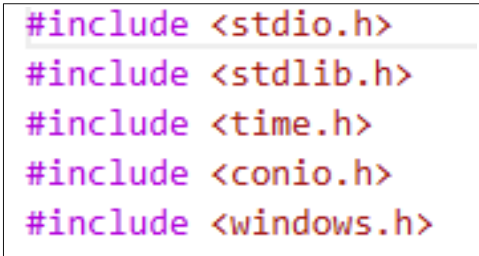
Для проверки стабильности работы программы выполним многократные запуски различных операций с разными размерами матриц и способами заполнения. Результаты тестирования должны показать, что программа корректно выполняет все предусмотренные функции и обрабатывает основные ошибки ввода данных.

Таким образом, проведённые тестирование подтвердят работоспособность разработанной программы. Все основные операции над матрицами будут работать корректно, а реализованные проверки позволят предотвратить выполнение неверных действий со стороны пользователя.

3 РАЗРАБОТКА ПРОГРАММЫ

В процессе выполнения курсовой работы была разработана консольная программа «Калькулятор матриц», предназначенная для выполнения основных операций над матрицами. Программа реализована на языке программирования С и работает в текстовом режиме [1]. Основной задачей при разработке являлось создание простой и понятной структуры программы с разделением операций на отдельные функции.

Во время разработки программы использовались стандартные библиотеки языка С, а также дополнительные библиотеки для работы с клавиатурой и консолью. В программе были подключены библиотеки `stdio.h`, `stdlib.h`, `time.h`, `conio.h` и `windows.h`, как указано на рисунке 3.1.



```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <conio.h>
#include <windows.h>
```

Рисунок 3.1 — Используемые библиотеки

Библиотека `stdio.h` использовалась для организации ввода и вывода данных. Из данной библиотеки применялись функции `printf()` и `scanf()` [4]. Функция `printf()` использовалась для вывода текста, меню и результатов вычислений на экран. Функция `scanf()` применялась для ввода размеров матриц, элементов матриц и скаляра [4].

Библиотека `stdlib.h` использовалась для работы с функцией `system()`, которая применялась для очистки экрана при помощи команды `system("cls")`. Также данная библиотека содержит функции, связанные с генерацией случайных чисел.

Библиотека `time.h` использовалась для получения текущего времени при помощи функции `time(NULL)`. Данное значение применялось в функции `srand()` для инициализации генератора случайных чисел.

Библиотека `conio.h` использовалась для работы с функцией `_getch()`. С её помощью реализовано управление меню при помощи клавиш клавиатуры без необходимости нажатия Enter после каждого действия.

Библиотека `windows.h` использовалась для подключения функции `SetConsoleOutputCP(CP_UTF8)` [6]. Данная функция необходима для корректного отображения символов в консоли Windows.

При разработке программы было принято решение разделить основные операции на отдельные функции, как показано на рисунке 3.2. Такой подход позволяет сделать код более структурированным и упрощает изменение программы в дальнейшем. Каждая функция выполняет только одну задачу,

благодаря чему повышается читаемость программы.

```
void info()...
void pause_screen()...
void slozhenie(int a[MAX][MAX], int b[MAX][MAX], int c[MAX][MAX], int n, int m)...
void vichitanie(int a[MAX][MAX], int b[MAX][MAX], int c[MAX][MAX], int n, int m)...
void umnozhenie(int a[MAX][MAX], int b[MAX][MAX], int c[MAX][MAX], int n, int m, int k)...
void transponirovanie(int a[MAX][MAX], int c[MAX][MAX], int n, int m)...
void umnozhenie_skalar(int a[MAX][MAX], int c[MAX][MAX], int n, int m, int s)...
void print(int a[MAX][MAX], int n, int m)...
void create_matrix(int a[MAX][MAX], int n, int m)...
int read_size(const char *name, int *r, int *c)...
int menu()...
int main()...
```

Рисунок 3.2 — Разделение на функции

Одной из первых была разработана функция `info()`. Она предназначена для вывода информации о программе. В данной функции отображается название курсовой работы, список операций и элементы управления. Функция не принимает входных параметров и не возвращает значение. Принцип работы функции заключается в последовательном выводе строк на экран при помощи функции `printf()` [4]. После отображения информации программа ожидает нажатия клавиши пользователем. При разработке данной функции было принято решение вынести стартовое окно в отдельный блок, чтобы не перегружать главную функцию программы большим количеством текста.

Следующей была реализована функция `pause_screen()`. Данная функция используется для временной остановки программы до нажатия клавиши пользователем. Она не принимает параметров и не возвращает значение. Необходимость данной функции связана с тем, что после выполнения операции результат может быстро исчезнуть при переходе к следующему экрану. Использование отдельной функции позволило избежать повторения одинакового кода в разных частях программы.

Для выполнения операций над матрицами были разработаны отдельные функции. Это позволило разделить вычислительную часть программы и интерфейс пользователя. Функция `slozhenie()` предназначена для сложения двух матриц. Входными параметрами функции являются две исходные матрицы, результирующая матрица, а также количество строк и столбцов. Результат выполнения операции записывается в третью матрицу. Алгоритм функции основан на использовании двух вложенных циклов [4]. Первый цикл перебирает строки матрицы, а второй — столбцы, как указано на рисунке 3.3. Для каждого элемента выполняется сложение соответствующих элементов двух матриц.

```
{  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < m; j++)  
            c[i][j] = a[i][j] + b[i][j];  
}
```

Рисунок 3.2 — Функция сложения

Функция `vichitanie()` работает аналогичным образом, но вместо сложения выполняется вычитание элементов матриц. Использование отдельной функции для вычитания позволило сделать структуру программы более понятной и упростило вызов операций из главного меню.

Одной из наиболее сложных функций программы является `umnozhenie()`, предназначенная для умножения матриц. Данная функция принимает две исходные матрицы, результирующую матрицу и размеры матриц. Для реализации алгоритма умножения использовались три вложенных цикла. Первый цикл отвечает за перебор строк первой матрицы, второй — за перебор столбцов второй матрицы, а третий выполняет вычисление суммы произведений элементов. При разработке функции особое внимание было уделено правильности вычислений и проверке размеров матриц перед выполнением операции. Использование трёх вложенных циклов соответствует стандартному алгоритму умножения матриц [5] и позволяет корректно выполнять вычисления для различных размеров.

Функция `umnozhenie_skalar()` предназначена для умножения матрицы на число. Входными параметрами являются исходная матрица, результирующая матрица, размеры матрицы и значение скаляра. Алгоритм работы функции достаточно простой. С помощью двух вложенных циклов выполняется последовательное умножение каждого элемента матрицы на заданное число.

Для транспонирования матрицы была разработана функция `transponirovanie()`. Её задачей является замена строк матрицы столбцами. В процессе работы функции используется двойной цикл. Элемент исходной матрицы копируется в новую матрицу с изменением индексов. При разработке функции было принято решение создавать отдельную результирующую матрицу, чтобы избежать ошибок при изменении исходных данных.

Функция `print()` используется для вывода матрицы на экран. Она принимает матрицу и её размеры. Внутри функции используются вложенные циклы, которые последовательно выводят все элементы матрицы. Данная функция была создана для того, чтобы избежать повторения одинакового кода вывода в разных частях программы. Кроме этого, использование отдельной функции упрощает изменение формата вывода данных.

Для создания матриц была разработана функция `create_matrix()`. Она является одной из основных частей программы, так как отвечает за заполнение матриц различными способами. Функция принимает матрицу и её размеры. Внутри функции реализовано дополнительное меню выбора способа создания матрицы. Пользователь может выбрать ручной ввод, создание нулевой матрицы,

единичной матрицы или случайное заполнение.

При реализации меню использовалась функция `_getch()`, позволяющая получать нажатия клавиш без ожидания Enter. Для обработки клавиш вверх и вниз применялись специальные коды клавиатуры [6].

Если пользователь выбирает ручной ввод, программа последовательно запрашивает значения элементов матрицы. При выборе нулевой матрицы все элементы получают значение ноль. При создании единичной матрицы программа выполняет дополнительную проверку. Если количество строк и столбцов не совпадает, выводится сообщение об ошибке. Такое решение было принято из-за того, что единичная матрица может быть только квадратной. Для случайного заполнения используется функция `rand() % 10`, которая создаёт случайные числа от 0 до 9 [4].

Следующей важной частью программы является функция `read_size()`. Она предназначена для ввода размеров матриц и проверки корректности данных. Функция принимает имя матрицы и адреса переменных, в которые будут записаны размеры. Если пользователь вводит размеры меньше или равные нулю либо превышающие установленное ограничение MAX, программа выводит сообщение об ошибке. Наличие данной функции позволило централизовать проверку размеров матриц и избежать повторения одинакового кода.

Для организации интерфейса программы была разработана функция `menu()`. Она отвечает за отображение главного меню и выбор пользователем необходимой операции. Внутри функции используется массив строк, содержащий названия пунктов меню. Текущая позиция курсора изменяется при нажатии клавиш вверх и вниз. После нажатия Enter функция возвращает номер выбранного действия. Такой способ реализации был выбран из-за удобства работы с меню и возможности быстро переключаться между операциями.

Главной частью программы является функция `main()`. В ней выполняется запуск программы и объединение всех остальных функций. После запуска программы вызывается функция `info()`, затем создаются массивы для хранения матриц и переменные для хранения размеров. Для организации основной работы программы используется бесконечный цикл `while(1)`. Внутри цикла вызывается функция `menu()`, после чего программа определяет выбранное действие пользователя при помощи оператора `switch`, как на рисунке 3.3.

```
while (1)
{
    vibor = menu();
    switch (vibor)
    {
```

Рисунок 3.3 — Бесконечный цикл со switch

Для каждой операции предусмотрен отдельный блок кода. В данных блоках выполняется ввод размеров матриц, создание матриц, проверка корректности данных и вызов соответствующей функции обработки. Если пользователь выбирает пункт выхода, программа завершает выполнение при помощи оператора `return 0`, что показано на рисунке 3.4.

```
case 6:
    system("cls");
    printf("Exit...\n");
    return 0;
}
```

Рисунок 3.4 — Выход из программы

Во время разработки программы большое внимание уделялось обработке ошибок пользователя. Были реализованы проверки размеров матриц, проверка совместимости матриц при выполнении операций и проверка возможности создания единичной матрицы.

После завершения разработки программа была протестирована на различных примерах.

Таким образом, в ходе разработки была создана консольная программа для выполнения операций над матрицами. Программа содержит отдельные функции для выполнения математических операций, организации интерфейса и проверки данных. Использование модульной структуры позволило сделать код более понятным и удобным для дальнейшего изменения.

4 ТЕСТИРОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ

После завершения разработки программы было проведено тестирование всех основных функций. Основной целью тестирования являлась проверка корректности выполнения операций над матрицами, проверка работы пользовательского интерфейса и обработка возможных ошибок при вводе данных. Во время проверки использовались полностью корректные, частично корректные и полностью неверные тестовые примеры.

Тестирование программы проводилось поэтапно. Сначала проверялась работа меню и интерфейса, после чего отдельно тестировались математические операции и обработка ошибок пользователя. Все проверки выполнялись в консольном режиме.

Во время тестирования главного меню (указан на рисунке 4.1) проверялась корректность перемещения между пунктами при помощи клавиш клавиатуры. Пользователь запускал программу и переходил между разделами меню клавишами вверх и вниз. После выбора пункта клавишей Enter программа открывала нужную операцию. В результате тестирования было установлено, что меню работает корректно и правильно обрабатывает действия пользователя.

```
=== MATRIX CALCULATOR ===  
  
-> slozhenie  
    vichitanie  
    umnozhenie  
    transpose  
    umnozhenie na skalar  
    exit  
|
```

Рисунок 4.1 — Главное меню

Также была проверена работа стартового окна программы, указан на рисунке 4.2. После запуска на экран выводилась информация о проекте, список поддерживаемых операций и элементы управления. После нажатия клавиши программа переходила в главное меню. Ошибок во время проверки не было обнаружено.

```
=====
MATRIX CALCULATOR
=====

Course project:
"Calculator for matrix operations"

Author: Shreider Anton
Group: E-64/25

Program functions:
- addition of matrices
- subtraction of matrices
- multiplication of matrices
- transpose
- multiplication by scalar

Controls:
UP / DOWN - move menu
ENTER - select

Press any key...
```

Рисунок 4.2 — Стартовое окно

Следующим этапом стало тестирование функции ввода размеров матриц. Для проверки использовались различные варианты данных. В полностью корректном случае пользователь вводил положительные размеры матриц, не превышающие установленное ограничение. Программа успешно принимала данные и переходила к следующему этапу работы.

В частично корректном примере пользователь вводил одно корректное значение и одно неверное. Например, количество строк было положительным, а количество столбцов равнялось нулю, как на рисунке 4.3. В таком случае программа выводила сообщение об ошибке и не позволяла продолжить выполнение операции.

```
Size of matrix A (rows cols): 5
0
Error: wrong size

Press any key...
```

Рисунок 4.3 — Защита от неверных размеров

В полностью неверных примерах, как на рисунке 4.4, вводились отрицательные значения или размеры, превышающие ограничение MAX. Программа выводила сообщение о неверном размере матрицы. Проверка показала, что функция контроля размеров работает правильно.

```
Size of matrix A (rows cols): 56  
-3  
Error: wrong size  
  
Press any key...|
```

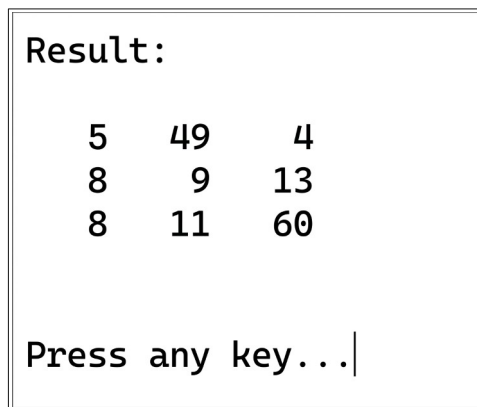
Рисунок 4.4 — Защита от неверных размеров

После этого проводилось тестирование функции создания матриц. Меню продемонстрировано на рисунке 4.5. Проверялись все способы заполнения матрицы: ручной ввод, создание нулевой матрицы, создание единичной матрицы и случайное заполнение (рисунок 4.5).

```
=== CREATE MATRIX ===  
  
-> manual  
    zero  
    identity  
    random  
|
```

Рисунок 4.5 — Меню создания матрицы

При ручном вводе пользователь последовательно вводил значения элементов матрицы. После завершения ввода программа корректно сохраняла данные и использовала их при выполнении операций. Во время тестирования ошибок не было обнаружено. Результат указан на рисунке 4.6



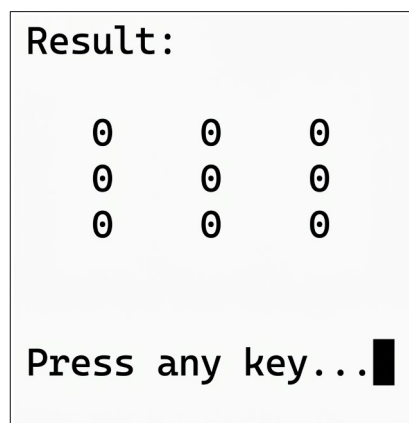
```
Result:

  5   49   4
  8    9  13
  8   11  60

Press any key...|
```

Рисунок 4.6 — Результат сложения

При создании нулевой матрицы, указана на рисунке 4.7 программа автоматически заполняла все элементы значением ноль. Проверка показала, что функция корректно работает для матриц различных размеров.



```
Result:

  0    0    0
  0    0    0
  0    0    0

Press any key...■
```

Рисунок 4.7 — Нулевая матрица

Во время тестирования единичной матрицы использовались как квадратные, так и прямоугольные матрицы. В корректном примере пользователь вводил одинаковое количество строк и столбцов. Программа успешно создавала единичную матрицу на рисунке 4.8, в которой элементы главной диагонали были равны единице, а остальные элементы — нулю .

```
Result:

  1    0    0
  0    1    0
  0    0    1

Press any key...|
```

Рисунок 4.8 — Правильная единичная матрица

В частично корректном случае пользователь выбирал создание единичной матрицы для прямоугольной таблицы, что показано на рисунке 4.9. Программа выводила сообщение о невозможности выполнения операции. Это показывает, что проверка матрицы работает правильно.

```
Only NxN matrix!

Press any key...|
```

Рисунок 4.9 — Неверная единичная матрица

При случайном заполнении матрицы программа генерировала случайные числа от 0 до 9. Во время тестирования проверялось заполнение матриц разных размеров. Результаты на рисунке 4.10 показали корректную работу генерации случайных значений.

```
Result:

  1    2    1
  3    1    9
  0    9    9

Press any key...|
```

Рисунок 4.10 — Верная случайная матрица

После проверки создания матриц выполнялось тестирование операции сложения. Для полностью корректного примера использовались две матрицы одинакового размера. Программа выполняла сложение соответствующих элементов и выводила результат на экран. Проверка вычислений, которая указана на рисунке 4.11 показала правильность работы алгоритма.

```
Result:

  13    4    9
  11   15    7
  10    3    1

Press any key...|
```

Рисунок 4.11 — Сложение матриц

В частично корректном примере использовались матрицы разных размеров. Программа обнаруживала несовпадение размеров и выводила сообщение об ошибке. Операция сложения не выполнялась.

В полностью неверном примере пользователь вводил недопустимые размеры матриц. Программа не переходила к вычислениям и сообщала о неверном вводе данных. Результаты тестирования показали, что функция сложения корректно работает как при правильных, так и при ошибочных данных.

Аналогичным образом проверялась операция вычитания матриц. В полностью корректных примерах программа правильно вычитала элементы второй матрицы из первой и выводила результат, как показано на рисунке 4.12. При несовпадении размеров программа выводила сообщение об ошибке и блокировала выполнение операции.

```
Result:

  1    3    3   -5
 -3    0    3    2
  4   -2   -5    3

Press any key...
```

Рисунок 4.12 — верный подсчет разницы

Во время тестирования операции умножения матриц особое внимание уделялось проверке совместимости размеров. В полностью корректном примере количество столбцов первой матрицы совпадало с количеством строк второй матрицы [5]. Программа успешно выполняла вычисления и выводила результирующую матрицу, как на рисунке 4.13.

```
Result:

54  48  42  24  54
22  18  16  26  30
28  21  19  53  48
31  26  23  30  39

Press any key...|
```

Рисунок 4.13 — Верная операция умножения

Также проверялась работа алгоритма умножения при разных размерах матриц, как указано на рисунке 4.14. Результаты тестирования подтвердили корректную работу трёх вложенных циклов, используемых для вычислений [2].

```
Size of matrix A (rows cols): 3
4
Size of matrix B (rows cols): 3
4
Error: columns of A != rows of B

Press any key...
```

Рисунок 4.14 — Неверная операция умножения

В частично корректном примере и размеры матриц не позволяли выполнить умножение. Программа обнаруживала ошибку и выводила сообщение о несовместимости размеров. Операция не выполнялась.

Далее проводилось тестирование функции транспонирования. Для проверки использовались матрицы различных размеров. После выполнения операции строки матрицы становились столбцами. Результаты, указанные на рисунке 4.15, тестирования показали, что функция корректно меняет индексы элементов и правильно формирует новую матрицу.

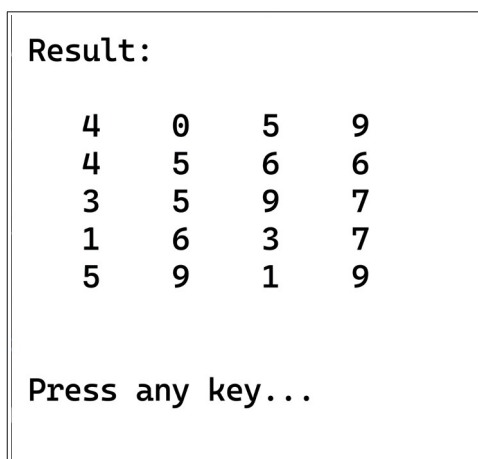


Рисунок 4.15 — Верная операция транспонирования

Также проверялись случаи с квадратными и прямоугольными матрицами. Во всех корректных примерах операция выполнялась правильно, что показано на рисунке 4.16. Ошибок в работе функции не было обнаружено.

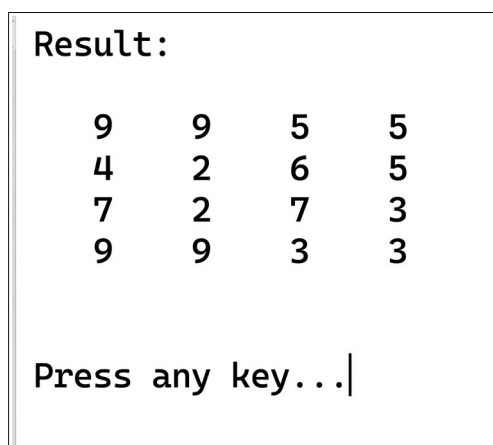
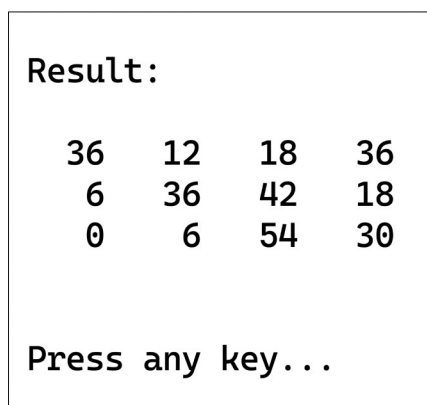


Рисунок 4.16 —Квадратная транспонированная матрица

После этого тестировалась функция умножения матрицы на скаляр. Пользователь вводил матрицу и значение числа, на которое необходимо выполнить умножение. Программа последовательно умножала каждый элемент матрицы на введённое значение и выводила результат на экран, как на рисунке 4.17.



Result:

36	12	18	36
6	36	42	18
0	6	54	30

Press any key...

Рисунок 4.17 — Умножение матрицы на скаляр

Для проверки использовались положительные, отрицательные и нулевые значения скаляра. Во всех случаях программа корректно выполняла вычисления. Также были проверены различные размеры матриц. Ошибок во время тестирования обнаружено не было.

Дополнительно проверялась стабильность работы программы при многократном выполнении операций. Программа запускалась несколько раз подряд, выполнялись различные действия, использовались разные размеры матриц и способы заполнения. Проверка показала, что программа работает стабильно и не завершает работу с ошибками.

Во время анализа результатов было установлено, что все основные функции программы работают корректно в полностью верных тестовых примерах. Операции над матрицами выполняются правильно, а результаты вычислений соответствуют ожидаемым значениям.

В частично корректных случаях программа правильно определяет ошибки пользователя и не позволяет выполнять операции при неправильных данных. Это позволяет избежать неверных вычислений и повышает надёжность программы.

В полностью неверных случаях программа также корректно обрабатывает ошибки ввода. Проверка размеров матриц и условий выполнения операций предотвращает некорректную работу приложения.

Результаты тестирования показывают, что разработанная программа успешно выполняет поставленные задачи. Все основные функции работают правильно, интерфейс корректно реагирует на действия пользователя, а реализованные проверки позволяют избежать большинства ошибок при работе с программой.

Таким образом, проведённое тестирование подтвердило работоспособность программы «Калькулятор матриц». Разработанное приложение корректно выполняет операции над матрицами, обрабатывает ошибки пользователя и обеспечивает стабильную работу в различных режимах использования.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана консольная программа «Калькулятор матриц», предназначенная для выполнения основных операций над матрицами. Программа позволяет выполнять сложение, вычитание, умножение матриц, транспонирование и умножение матрицы на скаляр. Также в программе реализовано несколько способов создания матриц, включая ручной ввод, создание нулевой и единичной матриц, а также случайное заполнение.

В качестве языка программирования был выбран язык C [1]. Для разработки использовались библиотеки `stdio.h`, `stdlib.h`, `time.h`, `conio.h` и `windows.h`. Использование данных библиотек позволило реализовать ввод и вывод данных, работу с клавиатурой, генерацию случайных чисел и управление консольным интерфейсом программы.

Работа над проектом начиналась с анализа поставленной задачи и определения основных функций будущей программы. На начальном этапе были изучены основные операции над матрицами и продуман способ организации интерфейса пользователя. После этого была разработана структура программы и определены основные функции, необходимые для выполнения операций.

Далее создавались функции для работы с матрицами, затем была реализована система меню и обработка действий пользователя. После написания основных функций выполнялось тестирование программы и проверка корректности вычислений.

Основные проблемы были связаны с реализацией управления меню при помощи клавиатуры, проверкой размеров матриц и разработкой обработок ошибок от пользователя. Также определённые трудности возникали при реализации алгоритма умножения матриц, так как данная операция требует правильной работы нескольких вложенных циклов.

Для решения данных проблем использовались пошаговая проверка работы программы и тестирование отдельных функций. Ошибки исправлялись после анализа результатов работы программы и повторной проверки вычислений. Разделение программы на отдельные функции также упростило поиск и исправление ошибок.

Таким образом, поставленная цель курсовой работы была достигнута. В процессе выполнения проекта были получены практические навыки разработки программ на языке C, работы с функциями, массивами, циклами и консольным интерфейсом. Также была изучена организация структуры программы и обработка пользовательского ввода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Язык программирования С, Керниган, Б., Ритчи, Д.. [Электронный ресурс]. — Режим доступа: http://www.r-5.org/files/books/computers/languages/c/kr/Brian_Kernighan_Dennis_Ritchie-The_C_Programming_Language-RU.pdf. Дата доступа: 28.02.2026
2. Теория матриц, Гантмахер, Ф. Р.. [Электронный ресурс]. — Режим доступа: <https://lib.brsu.by/sites/default/files/books/%D0%93%D0%B0%D0%BD%D1%82%D0%BC%D0%B0%D1%85%D0%B5%D1%80%20%D0%A4.%D0%A0.%20-%20%D0%A2%D0%B5%D0%BE%D1%80%D0%B8%D1%8F%20%D0%BC%D0%B0%D1%82%D1%80%D0%B8%D1%86.pdf>. Дата доступа: 28.02.26
3. Матрица (математика). [Электронный ресурс]. — Режим доступа: [https://ru.wikipedia.org/wiki/%D0%9C%D0%B0%D1%82%D1%80%D0%B8%D1%86%D0%B0_\(%D0%BC%D0%B0%D1%82%D0%B5%D0%BC%D0%B0%D1%82%D0%B8%D0%BA%D0%B0\)](https://ru.wikipedia.org/wiki/%D0%9C%D0%B0%D1%82%D1%80%D0%B8%D1%86%D0%B0_(%D0%BC%D0%B0%D1%82%D0%B5%D0%BC%D0%B0%D1%82%D0%B8%D0%BA%D0%B0)). Дата доступа: 07.03.26
4. Руководство по языку программирования Си. [Электронный ресурс]. — Режим доступа: <https://metanit.com/c/tutorial/>. Дата доступа 07.03.26
5. Умножение матриц. [Электронный ресурс]. — Режим доступа: https://ru.wikipedia.org/wiki/%D0%A3%D0%BC%D0%BD%D0%BE%D0%B6%D0%B5%D0%BD%D0%B8%D0%B5_%D0%BC%D0%B0%D1%82%D1%80%D0%B8%D1%86. Дата доступа: 16.03.26
6. Классические API консоли и последовательности виртуальных терминалов. [Электронный ресурс]. — Режим доступа: <https://learn.microsoft.com/ru-ru/windows/console/classic-vs-vt>. Дата доступа: 30.03.26